PONTIFICIA UNIVERSIDAD CATÓLICA DEL

PERÚ FACULTAD DE CIENCIAS E INGENIERÍA

INGENIERÍA MECATRÓNICA

2025 - 2

AUTOMATIZACIÓN INDUSTRIAL INTELIGENTE B

INTELIGENCIA ARTIFICIAL

Contenido

1. Visión por Computadora **3**
1.1 Definición 3
2. Visión por Computadora utilizando Feature Extraction **4**
3. Visión por Computadora utilizando IA **4**
4. Redes Neuronales Convolucionales (CNN) **5**
Elementos de Redes Neuronales Convolucionales 5
Proceso de Entrenamiento 8
Evaluación del Modelo 9
Ejemplo de Función de Pérdida 10
5. Inteligencia Artificial en Industria 5.0 **11**
6. EXPERIENCIA GUIADA **12**
7. EXPERIENCIA PRÁCTICA (primera parte) **18**
Anexos **21**
Anexo 1: Código de división de datos 21
Anexo 2: Tabla de parámetros del objeto model 23
Anexo 3: Tabla de parámetros del objeto model 26

OBJETIVOS

- Introducir al alumno la definición de visión por computadora
- Familiarizar al alumno con inteligencia artificial basica
- Familiarizar al alumno con el uso de librerías de visión por computadora en Python.
- Familiarizar al alumno con el uso del protocolo S7.
- Desarrollar programas en *Visual Studio* utilizando repositorios de GitHub.

MATERIALES

- *Camara Web*
- Visual Studio Code IDE
- Python

1. Visión por Computadora

1.1 Definición

La visión por computadora es una disciplina dentro de la inteligencia artificial que se centra en la replicación de la capacidad humana de la vista mediante el uso de cámaras, algoritmos y computadoras. Su objetivo es permitir que las máquinas interpreten y comprendan el mundo visual de manera similar a como lo hacen los humanos. Esta disciplina abarca una amplia gama de tareas, desde la identificación y clasificación de objetos en imágenes hasta la interpretación de escenas complejas y el análisis de videos en tiempo real.

El campo de la visión por computadora ha experimentado un crecimiento significativo debido a los avances en la tecnología de sensores, el aumento en la capacidad de procesamiento y los desarrollos en algoritmos y técnicas de inteligencia artificial. Las aplicaciones de la visión por computadora son vastas y diversas, incluyendo la seguridad y vigilancia, la medicina, la automoción, la manufactura, el entretenimiento, y la robótica, entre otros.

Un aspecto fundamental de la visión por computadora es la adquisición de imágenes, que implica el uso de cámaras y otros dispositivos de captura de imágenes para obtener datos visuales. Estos datos se procesan mediante algoritmos para extraer información relevante, como formas, colores, texturas y patrones. Posteriormente, esta información se utiliza para realizar tareas específicas, como el reconocimiento de objetos, la detección de eventos, la segmentación de imágenes y la reconstrucción tridimensional.

2. Visión por Computadora utilizando Feature Extraction

La visión por computadora tradicional se basa en técnicas de extracción de características que permiten identificar y describir elementos importantes dentro de una imagen. Uno de los métodos más básicos es la detección de color, que utiliza el análisis de los valores de los píxeles para segmentar y reconocer objetos basados en su color. Otro enfoque común es la transformada de Hough, que se emplea para la detección de formas geométricas como líneas y círculos dentro de una imagen. Estas técnicas son parte de un conjunto más amplio conocido como extracción de características, que también incluye el uso de descriptores como SIFT (Scale-Invariant Feature Transform) y SURF (Speeded Up Robust Features). Estos descriptores permiten la detección y descripción de puntos de interés en una imagen, que luego se pueden utilizar para tareas de correspondencia y reconocimiento. Aunque estos métodos tradicionales han sido muy útiles y siguen siendo relevantes, presentan limitaciones en cuanto a su capacidad para manejar variaciones complejas en las imágenes y requieren un diseño manual de las características.

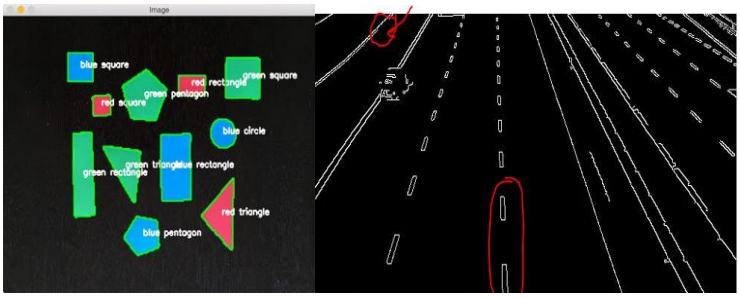


Figura 1: Visión por Computadora con Feature Extraction

3. Visión por Computadora utilizando IA

La evolución de la visión por computadora ha sido profundamente impactada por los avances en machine learning, especialmente con el surgimiento de las redes neuronales profundas (deep learning). En lugar de depender de características diseñadas manualmente, los métodos basados en machine learning permiten que los algoritmos aprendan directamente a partir de los datos. Las redes neuronales convolucionales (CNN) son especialmente efectivas para tareas de visión por computadora, ya que pueden aprender a identificar patrones y características relevantes en imágenes a través de múltiples capas de procesamiento. Estas técnicas han demostrado un rendimiento sobresaliente en tareas como el reconocimiento de objetos, la clasificación de imágenes, la detección de objetos y la segmentación semántica. Además, las redes neuronales profundas pueden manejar una gran variedad de variaciones en las imágenes, incluyendo

cambios en la iluminación, el ángulo y el contexto. El uso de datasets grandes y variados para entrenar estos modelos, junto con técnicas como el aprendizaje transferido, ha permitido avances significativos en la precisión y robustez de los sistemas de visión por computadora basados en machine learning.

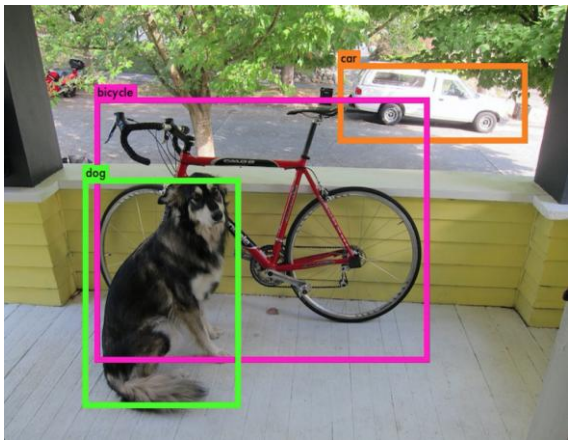


Figura 2: Visión por Computadora utilizando un modelo de Machine Learning

4. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN) son una clase de redes neuronales especialmente diseñadas para procesar y analizar datos con una estructura de rejilla, como imágenes. Se distinguen de las redes neuronales regulares (feedforward) por su arquitectura específica que incluye capas convolucionales, lo que les permite aprender características espaciales y jerárquicas directamente de los datos de entrada.

Elementos de Redes Neuronales Convolucionales

- Capas convolucionales: En estas capas, se aplican filtros (o kernels) que recorren la imagen de entrada para producir mapas de características. Cada filtro es capaz de detectar diferentes patrones, como bordes, texturas, o colores. El proceso de convolución implica multiplicar los valores de los píxeles de la imagen de entrada por los valores del filtro y sumar estos productos para obtener un único valor en el mapa de características. Este proceso se repite para cada posición del filtro sobre la imagen.

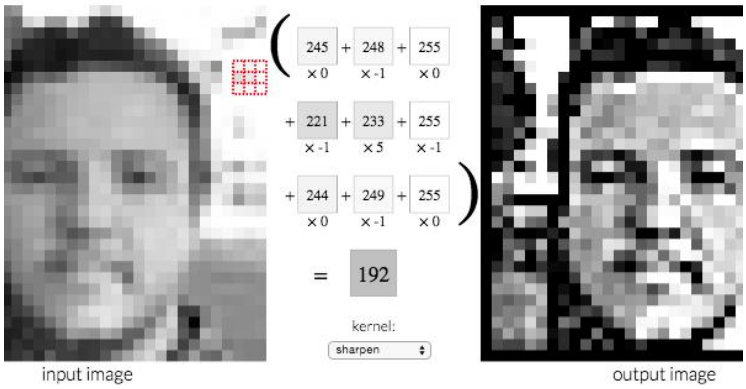


Figura 3: Efectos de la operación convolución

- Stride y Padding: El stride determina el paso con el que los filtros se mueven sobre la imagen, es decir, si el Stride es igual a 1, se moverá desplazándose una unidad como en la Figura 4. Por otro lado, el padding se usa para controlar el tamaño de la salida y conservar las dimensiones originales de la imagen.

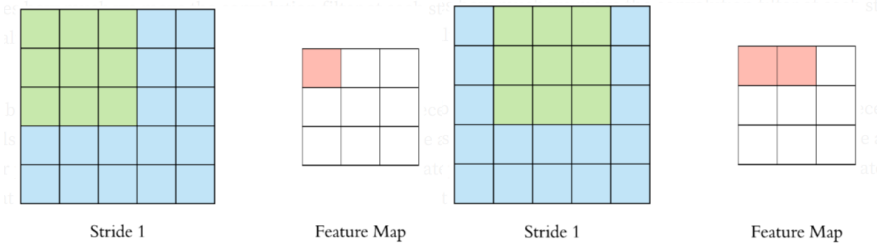


Figura 4: Efectos de la operación Stride y Padding

- Capas de Pooling: Max Pooling y Average Pooling: Estas capas reducen las dimensiones de los mapas de características, resumiendo la información más importante. Max pooling toma el valor máximo en una región, mientras que average pooling toma el promedio. Pooling ayuda a reducir el número de parámetros, prevenir el sobreajuste y disminuir el costo computacional, a la vez que conserva las características más relevantes de la imagen como se muestra en la Figura 5.

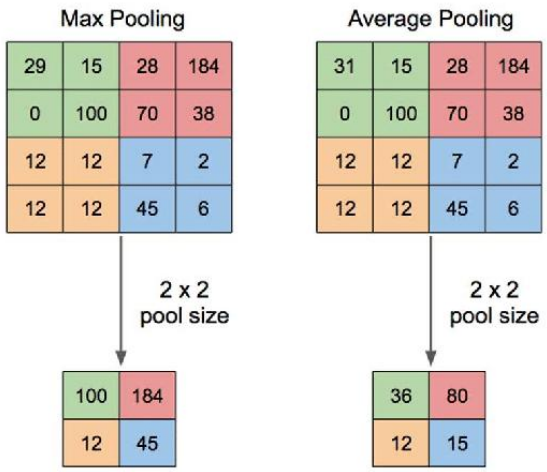


Figura 5: Efectos de la capa Pooling (Max y Average Pooling)

- Capas de Activación: Las funciones de activación introducen no linealidades en la red, permitiendo que aprenda representaciones complejas. Las más comunes son ReLU (Rectified Linear Unit), que aplica la función, y su variante Leaky ReLU, que permite pequeños valores negativos y ayuda a mitigar el problema de las unidades muertas en las redes profundas. En la Figura 6 se muestran más ejemplos de funciones. Estas funciones de activación son esenciales para que la red pueda aprender relaciones no lineales en los datos.

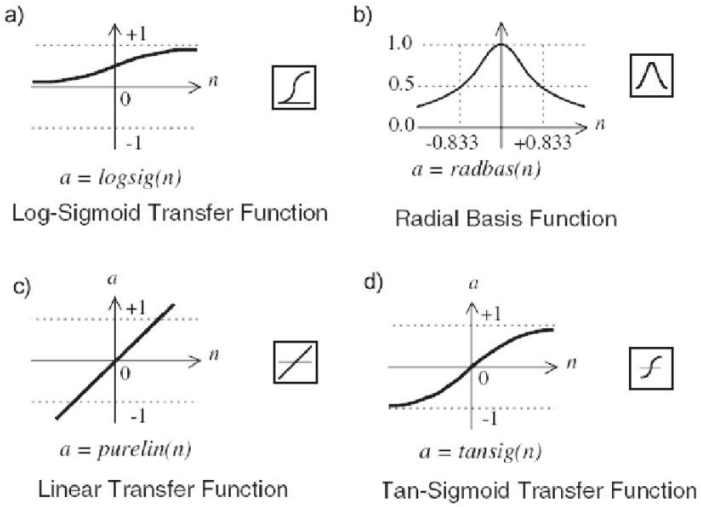


Figura 6: Funciones de activación

- Capas Completamente Conectadas (Fully Connected): Al final de la red, las capas completamente conectadas toman las características aprendidas y las combinan para realizar la tarea final, como la clasificación de imágenes. En estas capas, cada neurona está conectada a todas las neuronas de la capa anterior, lo que permite la

integración de toda la información extraída por las capas convolucionales. Estas capas son esenciales para la toma de decisiones basadas en las características aprendidas.

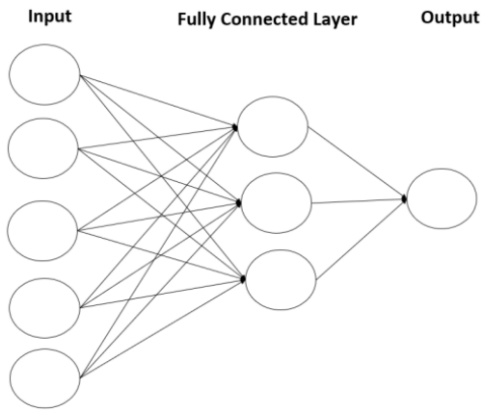


Figura 7: Capas Fully Connected

Proceso de Entrenamiento

1. Inicialización:

   Los pesos de la red (valores de los filtros) se inicializan, típicamente de manera aleatoria. La inicialización adecuada de los pesos es crucial para asegurar una convergencia rápida y efectiva durante el entrenamiento.

2. Propagación hacia Adelante (Forward Propagation):

   Como se muestra en la Figura 8, la imagen de entrada pasa a través de las capas convolucionales y de pooling, aplicando las funciones de activación y generando un conjunto de características. Este proceso transforma la imagen de entrada en una representación de características de mayor nivel que captura la información relevante para la tarea específica.

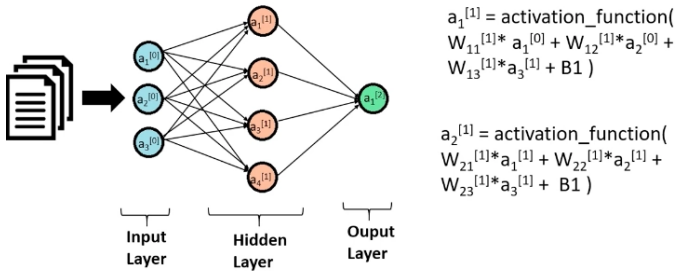


Figura 8: Forward Propagation

3. Cálculo de la Función de Pérdida:

   La función de pérdida mide la discrepancia entre las predicciones de la red y las etiquetas verdaderas. Ejemplos de funciones de pérdida incluyen la entropía cruzada para clasificación y el error cuadrático medio para regresión. Un ejemplo se muestra en la Figura 9.

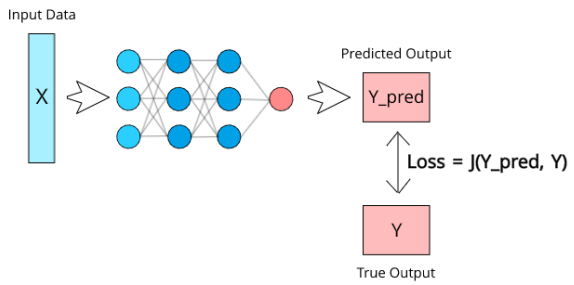


Figura 9: Uso de la función pérdida o Loss

4. Retropropagación (Backpropagation):

   Como se muestra en la Figura 10, este proceso ajusta los pesos de la red para minimizar la función de pérdida. Utiliza el gradiente de la pérdida con respecto a los pesos y aplica la regla de la cadena para actualizar los pesos en todas las capas.

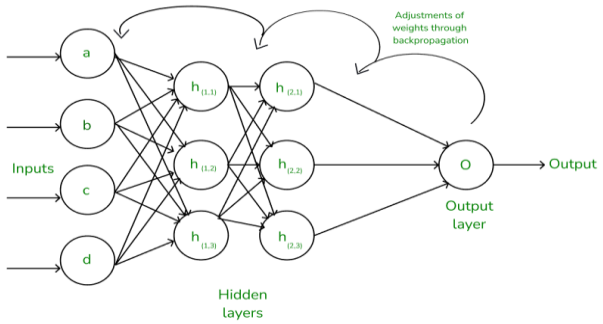


Figura 10: Backpropagation

5. Optimización:

   Los optimizadores son algoritmos que ajustan los pesos durante la retropropagación. Ejemplos comunes son el SGD (Stochastic Gradient Descent), Adam y RMSprop. Adam es particularmente popular por su capacidad de ajustar el tamaño del paso de aprendizaje adaptativamente.

Evaluación del Modelo

Conjunto de Validación:

Se utiliza un conjunto de datos separado del entrenamiento para ajustar los hiperparámetros y prevenir el sobreajuste. El conjunto de validación permite evaluar cómo el modelo generaliza los datos no vistos durante el entrenamiento.

Métricas de Rendimiento:

Para problemas de clasificación, las métricas comunes incluyen la precisión (accuracy), la precisión (precisión), el recall (recuerdo) y el F1-score. Para problemas de regresión, se utilizan el error cuadrático medio (MSE) y el coeficiente de determinación. Estas métricas proporcionan una evaluación cuantitativa del rendimiento del modelo en sus tareas específicas.

Evaluación en Conjunto de Prueba:

Se evalúa el modelo en un conjunto de prueba independiente para estimar su rendimiento en datos no vistos. Esta evaluación final proporciona una medida más precisa del rendimiento del modelo en un entorno del mundo real.

Ejemplo de Función de Pérdida

- **Entropía Cruzada**: La entropía cruzada es ampliamente utilizada en problemas de clasificación, ya que mide la divergencia entre las distribuciones de probabilidad predichas y las verdaderas.

$$\text{Loss} = -\sum_{i=1}^{\substack{\text{output}\\ \text{size}}} y_i \cdot \log \hat{y}_i$$

- **Error Cuadrático Medio (MSE):** El MSE es común en problemas de regresión, midiendo el promedio de los cuadrados de los errores entre las predicciones y los valores verdaderos.

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

5. Inteligencia Artificial en Industria 5.0

A lo largo de la historia, la industria ha pasado por distintas etapas de transformación. Desde la mecanización con la máquina de vapor en la Industria 1.0, pasando por la electricidad y la producción en masa en la 2.0, hasta llegar a la automatización con electrónica e informática en la 3.0, cada revolución ha cambiado nuestra forma de producir. La más reciente, la Industria 4.0, introdujo el uso intensivo de tecnologías como el Internet de las Cosas (IoT), la robótica avanzada, la computación en la nube y la Inteligencia Artificial (IA), enfocándose principalmente en automatizar procesos y maximizar la eficiencia.

Hoy estamos entrando en una nueva etapa conocida como **Industria 5.0**, que no busca únicamente mejorar la productividad, sino también integrar valores humanos, éticos y sostenibles en el entorno industrial. En lugar de ver a los humanos como piezas reemplazables en un sistema automatizado, la Industria 5.0 propone una colaboración estrecha entre humanos y máquinas, en la que cada uno aporta lo que mejor sabe hacer. Aquí, la Inteligencia Artificial juega un papel clave: no solo automatiza tareas, sino que se convierte en una herramienta que **potencia las capacidades humanas**.

La diferencia fundamental con la Industria 4.0 es el enfoque. Mientras que la anterior giraba en torno a la eficiencia técnica, **la Industria 5.0 pone al ser humano en el centro de los procesos**. Esto implica que la tecnología debe adaptarse a las personas, no al revés. La IA, en este contexto, se usa para asistir a los trabajadores en la toma de decisiones, personalizar productos de acuerdo con las necesidades del usuario, anticipar fallos antes de que ocurran, y proponer soluciones que respeten tanto los objetivos de producción como el bienestar social y ambiental.

Imagina una fábrica donde los operarios no están aislados de la tecnología, sino que trabajan en conjunto con sistemas inteligentes. Estos sistemas pueden interpretar instrucciones en lenguaje natural, entender gestos o incluso adaptarse al estado emocional del trabajador para brindar apoyo en tiempo real. Además, permiten una producción más personalizada: en lugar de fabricar en masa el mismo producto, es posible adaptar cada unidad según los requerimientos de cada cliente sin perder eficiencia. Esta combinación de precisión tecnológica y creatividad humana es lo que define a la Industria 5.0.

Desde el punto de vista académico y profesional, comprender este nuevo

paradigma es esencial. Como futuros ingenieros, científicos o desarrolladores, los estudiantes de pregrado deben prepararse no solo para diseñar sistemas eficientes, sino también para pensar en su impacto humano, ético y ambiental.

6. EXPERIENCIA GUIADA

En el presente capítulo se hará la implementación de un sistema para detección de botellas usando una arquitectura simple de Machine Learning. Los tipos de botellas a detectar o **targets** son botellas de plástico y botellas de vidrio. Para esto, es necesario acondicionar la data y ordenarla en carpetas como se muestra en la Figura. 11. Es recomendado dejar una gran parte de la data total para el entrenamiento (un 70% u 80% de la data total) y el restante para la validación y test si fuera necesario.

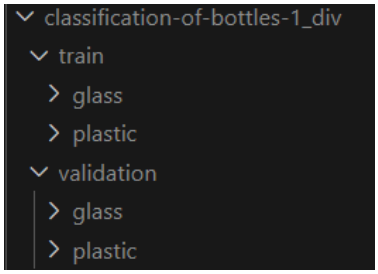


Figura 11: Organización del dataset

Lo siguiente es el diseño de la arquitectura a utilizar para la clasificación, para esto es recomendable realizar primero un esquema de cómo es la arquitectura como se muestra en la Figura 12.

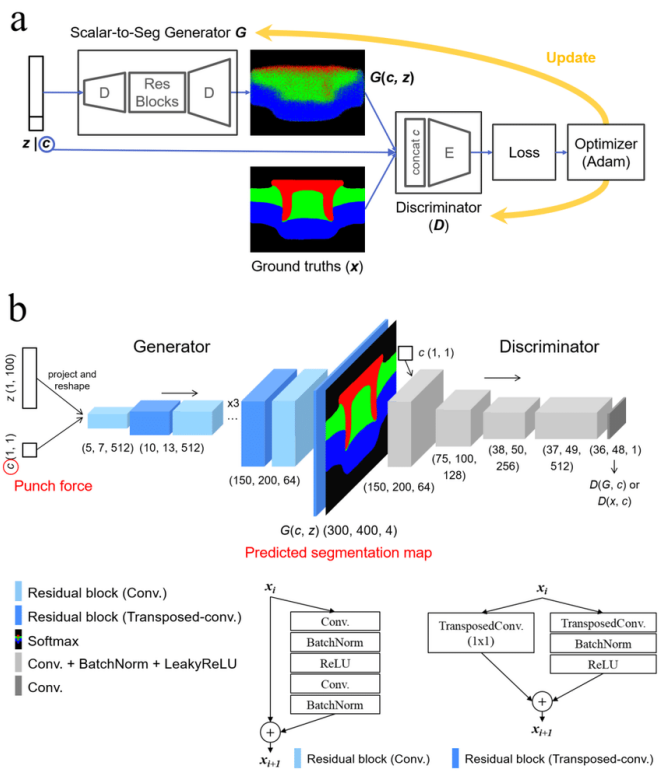


Figura 12: Ejemplo de arquitectura de Machine Learning

En nuestro caso dirigido, la arquitectura planteada para detección de tipo de botellas será más sencilla como se ilustra en la Figura 13. Como se puede observar únicamente hace uso de 2 capas de convolución (de dimensión 3x3 usando 32 filtros), 2 capas de max pooling (de dimensión 2x2) adicionando 2 capas fully connected (de dimensiones 128 y 2 para el dropout y la salida respectivamente). Es importante destacar que la entrada debe ser una imagen de dimensiones 64x64 pixeles y esta será ingresada como un dato de 3 canales debido a que es una imagen RGB.

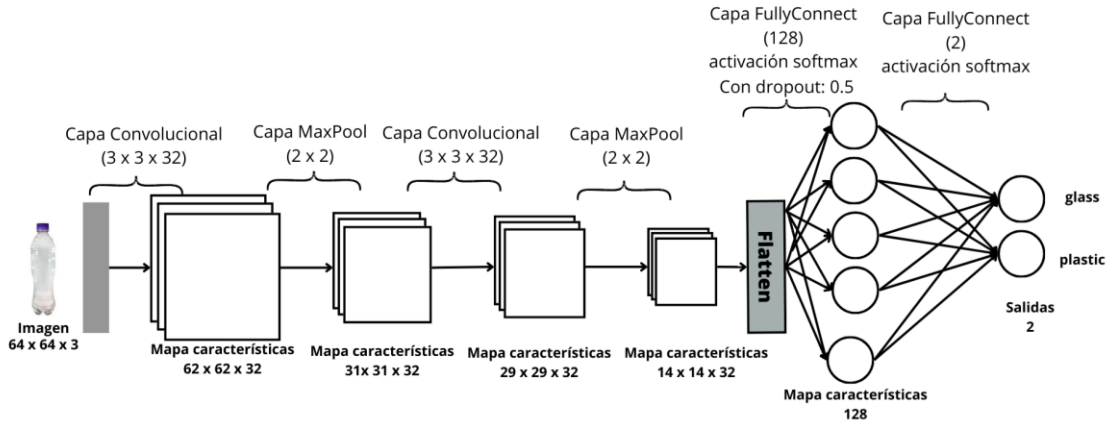


Figura 13: Arquitectura utilizada para el ejercicio guiado.

A continuación, será necesario definir de qué manera se compile el modelo o arquitectura diseñada, para esto es importante definir como mínimo 3 parámetros: El optimizador, la función de pérdida y la métrica a utilizar para la evaluación del modelo. Para este caso, al tratarse de una arquitectura de clasificación de imágenes se utiliza el modelo Adam, asimismo al tener más de dos salidas se utilizará la función de pérdida **CategoricalCrossentropy** y como métrica inicial se utilizará **accuracy**. Si se requiere utilizar otros tipos de optimizadores, funciones de pérdidas y métricas, puede encontrarlo en el Anexo 3.

Una vez planteado su arquitectura y la compilación, lo siguiente será implementarlo en código como se muestra en el Script 1. Se puede apreciar que primero es necesario importar las librerías necesarias para construir la arquitectura, siendo fundamental tener instalado las dependencias o librerias como tensorflow. De igual manera, al ser una arquitectura secuencial, esta se define inicialmente como tal para posteriormente realizar la construcción de la misma. Como se mencionó anteriormente, la arquitectura consta de **2 capas convolucionales**, **2 capas de max pooling** y **2 de fully connected** y estas se codifican con la sintaxis mostrada abajo, de requerir más información sobre los parámetros de entrada puede leer el Anexo 4. Finalmente, se realiza la compilación con los parámetros definidos anteriormente.

Script 1: Código para creación de arquitectura.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Crear el modelo
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(64, 64, 3)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(32, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(64, activation='relu'),
    Dense(2, activation='softmax')  # Para clasificación binaria (glass, plastic)
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

Una representación de manera más gráfica de cómo utilizar el código en base al esquema de la arquitectura se muestra en la Figura 14.

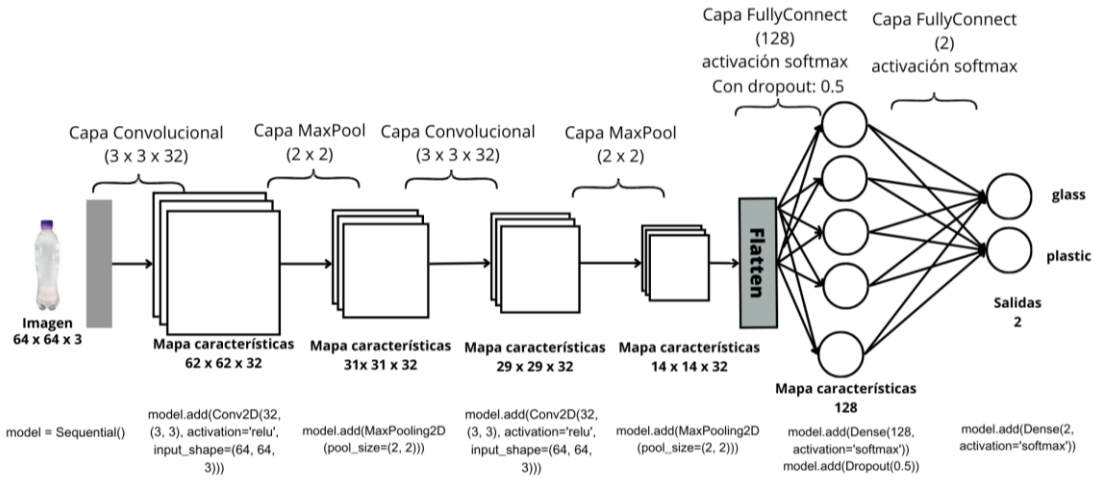


Figura 14: Arquitectura utilizada para el ejercicio guiado junto con código (Fila inferior).

Lo siguiente sería adaptar el tamaño de las imágenes en el caso de que no tengan las dimensiones requeridas por la arquitectura y adaptarlas para el entrenamiento del modelo. Asimismo es importante destacar que al preparase la data para ser entrenada se debe definir el tipo de clasificación que tendrá y el batch, el cual hace referencia al subconjunto de imágenes que se usará para entrenar por cada paso de cada época. El código para realizar dicha conversión se muestra en el Script 2

Script 2: Código para acondicionamiento del dataset de entrenamiento y validación

```
train_datagen = ImageDataGenerator(rescale=1./255)
validation_datagen = ImageDataGenerator(rescale=1./255)
output_dir = 'classification-of-bottles-1_div' # Aqui se debe colocar la dirección
del dataset dividido en train y validation
train_set = train_datagen.flow_from_directory(
    os.path.join(output_dir, 'train'),
    target_size=(64, 64), #Dimensiones requeridas
    batch_size=32,
    class_mode='categorical'
)

validation_set = validation_datagen.flow_from_directory(
    os.path.join(output_dir, 'validation'),
    target_size=(64, 64), #Dimensiones requeridas
    batch_size=32,
    class_mode='categorical'
)
```

A continuación se llevará a cabo el entrenamiento del modelo previamente construido. Para este proceso, es fundamental entender los parámetros utilizados durante el entrenamiento. El primer parámetro clave es la cantidad de épocas, que determina el número de veces que el modelo recorrerá el conjunto de datos completo durante el entrenamiento. A mayor número de épocas, el modelo tiene más oportunidades de ajustar sus pesos y mejorar su precisión, aunque un exceso de épocas puede llevar al sobreajuste.

Otro parámetro importante es el número de pasos por época, que define cuántos lotes de datos se procesan en cada época. Este valor está relacionado con el tamaño del conjunto de datos y el tamaño de los lotes (batches). Un tamaño de lote adecuado permite un equilibrio entre la eficiencia del entrenamiento y la estabilidad del gradiente, mientras que los pasos por época aseguran que se procesen todas las muestras disponibles en cada ciclo de entrenamiento.

Script 3: Código para entrenamiento del modelo de Machine Learning

```
model.fit(
    train_set,
    steps_per_epoch=train_set.samples // train_set.batch_size,
    epochs=100,
    validation_data=validation_set,
    validation_steps=validation_set.samples // validation_set.batch_size
)

# Evaluación
loss, accuracy = model.evaluate(validation_set)
print(f'Loss: {loss:.4f}, Accuracy: {accuracy:.4f}')
```

Se construye la matriz de confusión del modelo construido con el fin de conocer otras métricas si se desea como se muestra en el Script 4.

Script 4: Construcción de la matriz de confusión

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
predictions = model.predict(validation_set, verbose=1)
# Convertir probabilidades en etiquetas predichas
y_pred = tf.argmax(predictions, axis=1).numpy()
# Etiquetas verdaderas
y_true = validation_set.classes
# Obtener número de clases
num_classes = len(validation_set.class_indices)
# Crear matriz de confusión vacía
confusion_mtx = np.zeros((num_classes, num_classes),
dtype=np.int32)
# Rellenar matriz de confusión
for true_label, pred_label in zip(y_true, y_pred):
    confusion_mtx[true_label][pred_label] += 1
# Nombres de clases (ordenados según el generador)
class_names = list(validation_set.class_indices.keys())
# Graficar la matriz de confusión
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_mtx, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Matriz de Confusión (TensorFlow)")
plt.show()
```

Finalmente, se prueba el modelo con una imagen de prueba utilizando el siguiente código. Recordar que las variables dentro de **class_indices** deben estar en **orden alfabético.** En el caso de colores sería, por ejemplo, **{'amarillo':0, 'azul': 1, 'rojo':2}**

Script 5: Uso del modelo de Machine Learning para pruebas

```
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
# Diccionario de clases (ajustar si fue entrenado con más o
diferentes clases)
class_indices = {'glass': 0, 'plastic': 1}
class_labels = {v: k for k, v in class_indices.items()}  #
Invertimos para obtener nombre desde índice

def predict_image(img_path, model, class_labels):
    try:
        # Cargar y preprocesar la imagen
        img = image.load_img(img_path, target_size=(64, 64))
        img_array = image.img_to_array(img)
        img_array = np.expand_dims(img_array, axis=0)  # (1,
64, 64, 3)
        img_array /= 255.0

        # Hacer la predicción
        prediction = model.predict(img_array)
        predicted_index = np.argmax(prediction)
        predicted_label = class_labels[predicted_index]
        confidence = prediction[0][predicted_index]

        return f'Predicción: {predicted_label} (Confianza:
{confidence:.2f})'
    except Exception as e:
        return f'Error al procesar la imagen: {e}'

# Ruta de la imagen individual que deseas probar
img_path = #"DIRECCIÓN DE IMAGEN"
print(predict_image(img_path, model, class_labels))#Se imprime
el resultado
```

7. EXPERIENCIA PRÁCTICA

Una empresa dedicada al empaquetado de juguetes ha identificado cuatro tipos de fallas en sus productos, las cuales están representadas mediante tapitas de diferentes colores. Como ingeniero de automatización, tu tarea es desarrollar un sistema que clasifique correctamente estos defectos para facilitar su recolección y manejo en el proceso productivo. Para lograrlo, se entrenarán modelos de Machine Learning que permitan detectar las tapitas de colores mostradas en la figura adjunta.

Para este ejercicio, es necesario contar con una computadora que tenga instalado el intérprete de Python y las librerías requeridas para el desarrollo. El primer paso fundamental es obtener y preparar los datos que se utilizarán para el entrenamiento, la validación y la prueba del algoritmo de clasificación. **Usar Anexo 4**

Actividad 1: Obtención y separación de datos (4 puntos)

Para esta actividad será necesario tomar fotos o un video de cada color de tapa así como del fondo donde se encontrarán las tapas, siendo este la faja utilizado en laboratorios anteriores. **Cabe mencionar que mientras más datos se usen, la eficiencia de la red neuronal se incrementa.** Como se muestra en la Figura 15, en el caso se use vídeos **estos vídeos deben estar en la misma carpeta o dirección** que el código **DividirVideo.py** el cual sirve para separar vídeos en imágenes como se muestra en el Anexo 1.

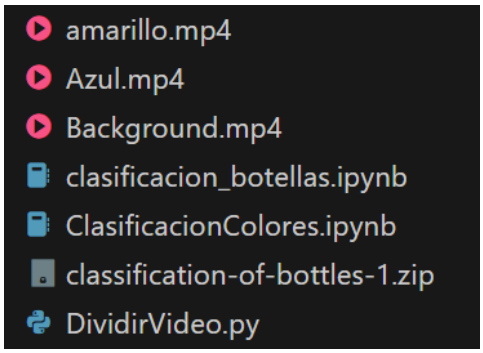


Figura 15: Videos en la misma carpeta que el código DividirVideo.py.

Posteriormente, se tendrá que realizar un código para separar la data en entrenamiento y validation como se muestra en la Figura 16. El porcentaje de imagenes que deben haber en train es del 80% de cada tipo de color mientras que en el validation es de 20%. Es recomendable que la cantidad de imágenes de cada color sea el mismo para evitar sobreajuste, sin embargo no es obligatorio para esta parte. Para realizar esta parte pueden guiarse del Anexo 2.

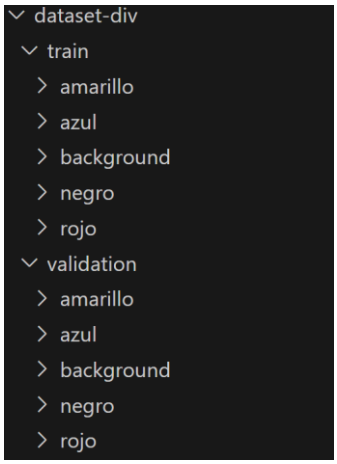


Figura 16: Dataset dividido.

Actividad 2: Importación de librerías y creación de la arquitectura (7 puntos)

En esta parte de la experiencia se les solicita completar algunas partes faltantes del Notebook **"Clasificacion_tapas_alumnos.ipynb".** Como primer paso, el código tiene una serie de parámetros generales al incio:

```
"""
Parametros generales
"""

IMG_SIZE = #Tamaño de imagenes preprocesadas
NUM_CLASSES =  # Numero de clases
BATCH_SIZE = 32
EPOCHS = 20
LEARNING_RATE = # Learning rate
AUG_PER_IMAGE = # Definir aumento de datos

CLASS_NAMES = ["CLASE1", "CLASE2", "CLASE3", "CLASE4"] #COMPLETAR con nombres de las carpetas dentro de dataset/
```

Se les da como recomendaciones utilizar un **tamaño de imagen de 32 o 64**, un **learning rate de 0.0001** y un **aumento de datos entre 3 a 6**. Tanto el tamaño de imagen y el aumento de datos son parámetros clave que pueden modificar en un futuro para optimizar el rendimiento de su modelo. También en la variable **CLASS_NAMES** se les solicita completar los nombres de sus clases a clasificar, tal cual se hayan generado con el **DividirVideo.py.**

Se les pide completar la pregunta luego de correr esta celda **(Se tiene que responder tanto en el Notebook como en la FE**):

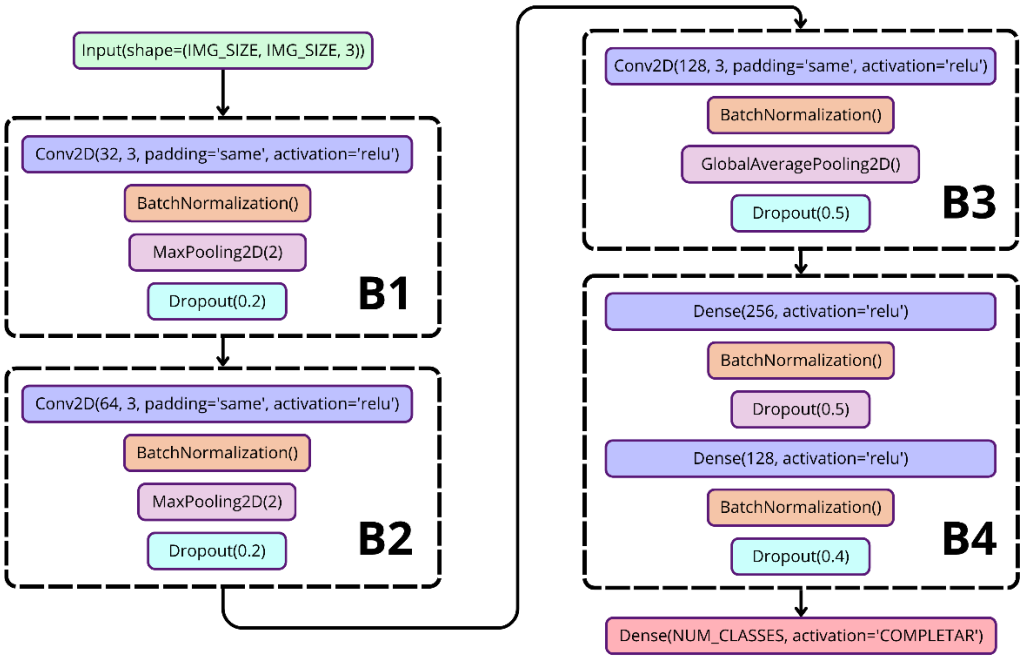
```
"""
PREGUNTA
"""

class_weights = compute_class_weights(y_train)

show_sample_images(X_train, y_train)

"""
¿Qué se puede observar de las imagenes mostradas? ¿Por qué salen de esa manera?
"""
```

En la sección de **“Definición de arquitectura”** se les pide replicar la siguiente arquitectura de la imagen:



Para la función de perdida a utilizar en el compilado del modelo, se les pide que sea la “`sparse_categorical_crossentropy`” y metrica durante el entrenamiento utilizar `'accuracy'`.

Actividad 3: Entrenamiento y análisis de rendimiento (3 puntos)

Para el entrenamiento se utiliza el **model.fit(),** esta celda ya se encuentra implementada, solo se tiene que ejecutar (El entrenamiento del modelo puede durar entre 5 – 15 min). Finalizado el entrenamiento, se tiene que correr la celda **"Métricas del modelo".** Esta celda imprimirá varias métricas útiles para la interpretación del modelo. Se le solicita realizar un análisis de la grafica de precisión del modelo, perdida del modelo, f1-score y matriz de confusión para poder saber si este modelo es optimo o si requiere algún ajuste adicional. Adicionalmente a ello, incluir cómo es que el modelo actual puede ser incorporado en su proyecto y en qué se relaciona con la Industria 5.0.

Actividad 4: Función para predicción (2 puntos)

El modelo fue entrenado con un normalizado y una reducción en la dimensión de las imágenes. Como las cámaras a utilizar tienen este ultimo valor variable, se tiene que realizar una función que garantice que el tamaño de las imágenes de entrada del modelo sean las mismas con las que se ha realizado el entrenamiento. Por lo que se les solicita completar esa sección de la función **predict_image(),** recordar que hay una variable que ya tiene ese valor.

```
img_resized = cv2.resize(img, ( COMPLETAR , COMPLETAR)) # <<< COMPLETAR >>>
```

Luego se busca realizar una prueba de funcionamiento, completar con la ruta a una imagen de prueba para validar el correcto funcionamiento del modelo.

```
predict_image("RUTA A IMAGEN.jpg", model)
```

Finalmente se tiene que guardar el modelo para su posterior uso, para ello, se utiliza la función **model.save()** presente en el código.

Actividad 5: Experimentos con video y mejoras (3 puntos)

Una vez ya se encuentra completamente entrenado el modelo, se busca poder probarlo en un entorno de aplicación real. Por ello se les pide que puedan cargar el modelo al archivo y actualizar los valores de los parámetros en el archivo **Prueba_video.py** y responder a las preguntas presentes dentro del código.

Finalmente se les pide variar los parámetros iniciales y volver a correr el Notebook de entrenamiento para ver si se pudo mejorar en algo los resultados con las mejoras propuestas.

Anexo 1: Tabla de parámetros del objeto model

En la presente tabla se encuentran los parámetros del objeto model. Recordar colocar los parámetros despues de un model, por ejemplo (model.Conv2D(32,(3,3))

Parámetro	Descripción	Cuándo se utiliza
SGD	**Gradiente descendente estocástico. Actualiza los pesos según la tasa de aprendizaje.**	**Se usa cuando quieres un algoritmo simple y eficiente, especialmente en problemas de redes profundas.**
Adam	**Algoritmo de optimización que combina la ventaja de AdaGrad y RMSProp.**	**Es el optimizador más usado en problemas complejos como redes neuronales profundas, NLP, y visión.**
RMSprop	**Divide el gradiente por una media móvil de las magnitudes de gradientes recientes.**	**Utilizado en modelos recurrentes o redes profundas, donde el gradiente cambia rápidamente.**
Adagrad	**Aumenta la tasa de aprendizaje para parámetros poco frecuentes y la reduce para los más comunes**	**Se utiliza cuando hay características esporádicas en el dataset (es decir, datos dispersos).**
Adadelta	**Extensión de Adagrad que reduce su tasa de aprendizaje decaída con el tiempo.**	**Usado en escenarios donde la tasa de aprendizaje de Adagrad disminuye demasiado rápido.**
Adamax	**Variante de Adam basada en la norma infinito.**	**Utilizado para redes profundas cuando Adam no es estable o para grandes lotes de datos.**
Nadam	**Adam con momento de Nesterov.**	**Ideal para problemas de optimización más sensibles, donde la convergencia puede ser más rápida.**

Función de pérdida	Descripción	Cuándo se utiliza
BinaryCrossentropy	Mide la diferencia entre la salida binaria predicha y los valores reales.	Utilizada en problemas de clasificación binaria (dos clases).
CategoricalCrossentropy	Calcula la pérdida entre la probabilidad de salida y la verdadera etiqueta en forma de one-hot.	Usada cuando las etiquetas de clase están en formato one-hot y hay más de dos clases.
SparseCategoricalCrossentropy	Similar a CategoricalCrossentropy, pero acepta etiquetas enteras en lugar de one-hot.	Se usa cuando las etiquetas no están en formato one-hot, sino como enteros, en problemas de clasificación.
MeanSquaredError (MSE)	Promedio de los errores cuadrados entre predicción y valor real.	Usado en problemas de regresión donde se minimiza la distancia cuadrática entre predicciones y valores.
MeanAbsoluteError (MAE)	Promedio de los errores absolutos entre predicciones y valores reales.	Utilizado en problemas de regresión donde se prefiere minimizar el error absoluto, como predicción de precios.
Huber	Combina MSE y MAE. Penaliza de forma cuadrática cuando los errores son pequeños y lineal cuando son grandes.	Usado en regresión robusta, donde se quiere minimizar el impacto de los outliers.
Hinge	Pérdida utilizada para problemas de clasificación binaria con margen máximo (SVM).	Se usa en modelos lineales y de soporte de vectores.
Poisson	Calcula la pérdida basada en la distribución de Poisson.	Usado en problemas donde los datos siguen una distribución de Poisson (como conteo de eventos).
CosineSimilarity	Mide la similitud entre dos vectores utilizando el coseno del ángulo entre ellos.	Utilizado cuando te interesa la dirección entre predicciones y objetivos más que su magnitud.

Métrica	Descripción	Cuándo se utiliza

Accuracy	**Proporción de predicciones correctas.**	**Usada en problemas de clasificación general, donde el objetivo es contar las predicciones correctas.**
BinaryAccuracy	**Precisión para problemas binarios.**	**Se usa cuando se tiene un problema de clasificación binaria (dos clases).**
CategoricalAccuracy	**Precisión para problemas de clasificación con múltiples clases.**	**Utilizado cuando las etiquetas de clase están en formato one-hot.**
SparseCategoricalAccuracy	**Precisión para problemas de clasificación con múltiples clases, sin etiquetas one-hot.**	**Utilizado cuando las etiquetas no están en formato one-hot, sino como enteros.**
TopKCategoricalAccuracy	**Mide si la clase verdadera está entre las K clases predichas con mayor probabilidad.**	**Se utiliza en problemas de clasificación donde no solo importa la primera opción, sino las K mejores opciones.**
MeanSquaredError (MSE)	**Error cuadrático medio entre las predicciones y los valores reales.**	**Usado en problemas de regresión, mide cuán cerca están las predicciones de los valores reales.**
MeanAbsoluteError (MAE)	**Error absoluto medio entre predicciones y valores reales.**	**Se usa en regresión, mide el error en términos absolutos, sin penalizar tanto los outliers.**
AUC	**Área bajo la curva ROC o PR. Mide qué tan bien las predicciones separan las clases.**	**Usado en problemas de clasificación, especialmente con desequilibrio de clases.**
Precision	**Proporción de verdaderos positivos entre todas las predicciones positivas.**	**Utilizado cuando te interesa cuán precisa es la predicción de la clase positiva (evitar falsos positivos).**
Recall	**Proporción de verdaderos positivos entre todos los casos positivos.**	**Se usa cuando quieres capturar todos los verdaderos positivos (evitar falsos negativos).**
F1Score	**Promedio armónico entre precisión y recall.**	**Ideal en situaciones donde existe un equilibrio entre precisión y recall.**

Anexo 3: Tabla de parámetros del objeto model

En la presente tabla se encuentran los parámetros del objeto model. Recordar colocar los parámetros despues de un model, por ejemplo (model.Conv2D(32,(3,3))

Parámetro	Posibles Opciones	Sintaxis
Conv2D - Filtros	**Número de filtros (kernels) en la capa convolucional: 32, 64, 128, 256, etc.**	**Conv2D(32)**
Conv2D - Tamaño del kernel	**Dimensiones del kernel (filtro): (3, 3), (5, 5), (7, 7), etc.**	**Conv2D(32, (3, 3))**
Conv2D - Función de activación	**Funciones de activación: 'relu', 'sigmoid', 'tanh', 'softplus', 'elu', 'selu', 'softsign', 'swish', etc.**	**Conv2D(32, (3, 3), activation='relu')**
Conv2D - input_shape	**Forma de entrada: (ancho, alto, canales) como (64, 64, 3) o cualquier tamaño de entrada de la imagen. Ejemplo: (128, 128, 3), (224, 224, 3) (solo para la primera capa convolucional).**	**Conv2D(32, (3, 3), input_shape=(64, 64, 3))**
MaxPooling2D - Tamaño del pool	**Tamaño de la ventana de pooling: (2, 2), (3, 3), (4, 4).**	**MaxPooling2D(pool_size=(2, 2))**
MaxPooling2D - Strides	**Tamaño de los pasos: (2, 2) (por defecto), (1, 1), (3, 3), etc.**	MaxPooling2D(pool_size=(2, 2), strides=(2, 2))
Flatten	**Sin parámetros, simplemente aplanamiento de las dimensiones del tensor.**	**Flatten()**
Dense - Unidades	**Número de neuronas (unidades) en la capa completamente conectada: 64, 128, 256, 512, etc.**	**Dense(128)**

CONTINÚA EN SIGUIENTE PÁGINA

Parámetro	Posibles Opciones	Sintaxis
Dense - Función de activación	**Funciones de activación: 'relu', 'sigmoid', 'tanh', 'softmax' (especialmente para la capa de**	**Dense(128, activation='relu')**

	salida), 'linear', etc.	
Dropout - Ratio	Proporción de desactivación de neuronas: 0.1, 0.2, 0.3, 0.5, etc.	Dropout(0.5)
Capa de salida - Unidades	Número de clases a predecir: 2 (para 2 clases: 'amarillo', 'rojo'), 3 (para 3 clases), etc.	Dense(2)
Capa de salida - Activación	Función de activación para la salida: 'softmax' (multiclase), 'sigmoid' (binaria).	Dense(2, activation='softmax')
Optimizador	Optimizadores del modelo: 'adam', 'sgd', 'rmsprop', 'adagrad', 'adamax', 'nadam', etc.	model.compile(optimizer='adam')
Pérdida (loss function)	Función de pérdida: 'categorical_crossentropy' (para más de 2 clases), 'binary_crossentropy' (para 2 clases), 'mean_squared_error', 'mean_absolute_error', 'sparse_categorical_crossentropy', etc.	model.compile(loss='categorical_crossentropy')
Métricas	Métricas para evaluar el rendimiento: ['accuracy'], ['precision'], ['recall'], ['f1_score'] (usualmente personalizada).	model.compile(metrics=['accuracy'])

Anexo 4: Creación de Entorno Virtual

ClasificacionColores.ipynb > Etapa de División de la data en Train y Validation

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python 3.10.4

Etapa de División de la data en Train y Validation

En esta etapa se divide la data de botellas en Train y Validation

```python
import os
import shutil
import random

def split_dataset(dataset_dir, output_dir, split_ratio=0.8):
    """
    Divide el conjunto de datos en entrenamiento y prueba.

    :param dataset_dir: Directorio que contiene las carpetas
    :param output_dir: Directorio donde se guardarán los datos divididos.
    :param split_ratio: Proporción de datos para entrenamiento (el resto será para prueba).
    """
    # Crear directorios de salida
    train_dir = os.path.join(output_dir, 'train')
    validation_dir = os.path.join(output_dir, 'validation')
```

Change kernel for 'ClasificacionColores.ipynb'

Python 3.10.4 ~\AppData\Local\Programs\Python\Python310\python.exe - Currently Selected

.venv (Python 3.10.4) .venv\Scripts\python.exe

Select Another Kernel...

Select Another Kernel

Type to choose a kernel source

Python Environments...

Existing Jupyter Server...

Select a Python Environment

Create Python Environment

.venv (Python 3.10.4) .venv\Scripts\python.exe — Recommended

anaconda3 (Python 3.9.7) ~\anaconda3\python.exe — Conda Env

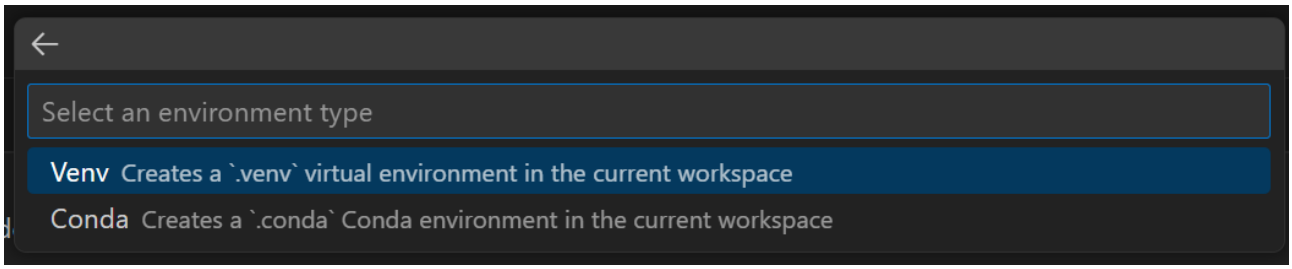
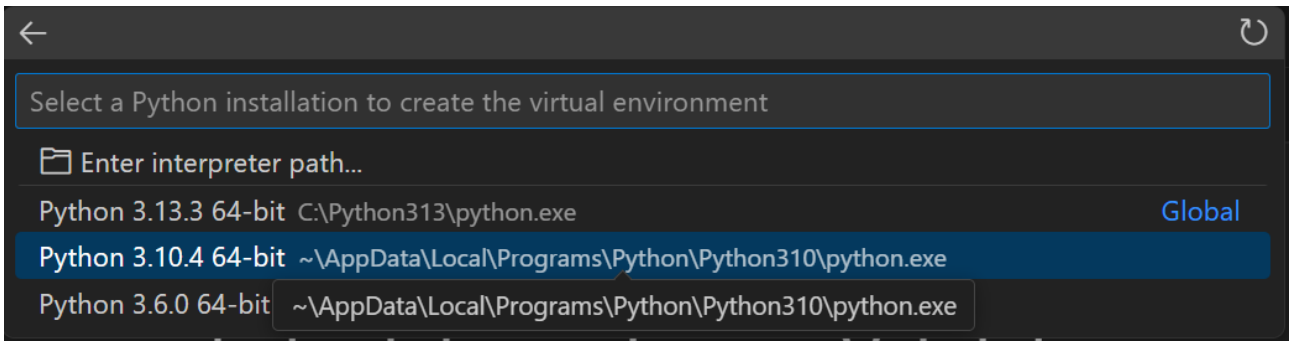
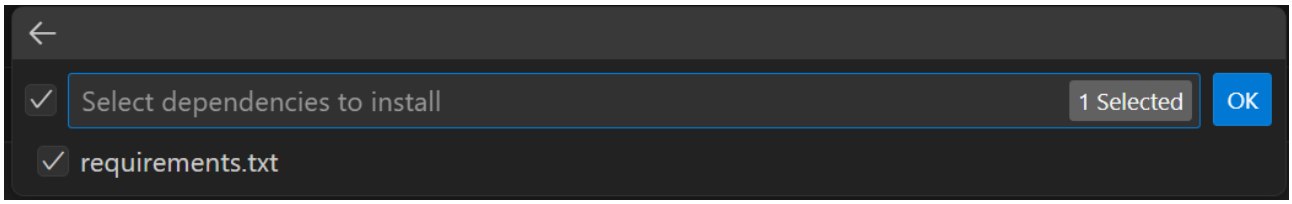
base (Python 3.6.5) c:\Program Files (x86)\Microsoft Visual Studio\Shared\Anaconda3_64\python.exe

lucid_env (Python 3.7.11) c:\ProgramData\Lucid Vision Labs\ArenaView\ArenaPy\Miniconda3\envs\luci...

Python 3.10.4 ~\AppData\Local\Programs\Python\Python310\python.exe — Global Env

Python 3.13.3 c:\Python313\python.exe

Python 3.6.0 ~\AppData\Local\Programs\Python\Python36\python.exe



Elegir siempre la versión de **64-BIT**